

Number: P3852R1

Date: 2026-05-12

Audience: [Library Evolution](#)

Target: C++29

Author: [Hana Dusíková](#)

- 
- [Changes](#)
  - [Motivation](#)
    - [restrict-like semantic](#)
    - [defining total ordering is not feasible](#)
    - [can we make `std::less<T\*>` work in constant evaluator](#)
    - [static analysis and contracts](#)
    - [constexpr `std::hive`](#)
  - [Proposed function](#)
    - [Why not just provide check for related pointers?](#)
    - [Why not span?](#)
    - [Why it's `\[first, last\)` and not inclusive range?](#)
    - [This signature seems error prone](#)
    - [Symmetry with `clamp`](#)
    - [std::less<T\\*>](#)
    - [Weird or segmented architectures](#)
  - [Implementation](#)
    - [Alternative way how to get same result](#)
  - [Wording](#)
    - [Feature test macro](#)

Thanks to Tomasz Kamiński for not letting me hit a painted tunnel on side of a wall.

## P3852R1: constexpr is\_pointer\_within memory area

---

Currently there is no way to check in a standard way if a pointer lies within specific memory area without running into implementation specific behaviour. During constant evaluation any comparison of pointers can lead to failure to constant evaluate in case the pointers are unrelated and points to different top-level objects.

This paper proposes functionality which **doesn't propose total ordering of pointers**. This paper allows you to check if the pointers are related and in a range, it needs compiler magic to do so.

### Changes

---

R0 → R1: added discussion about possibility on [some architectures to return false positive](#), [added precondition](#) ptr is not a past-the-end pointer. Rename the function from `is_pointer_between` to `is_pointer_within`.

# Motivation

---

At first maybe this does seem as somehow arcane or esoteric. But this is a problem worth solving and it will make C++ a bit more expressive.

## restrict-like semantic

---

Currently you can't express assumption that some memory span doesn't have anything in common with other memory span. With this proposal you can do it in portable and defined way.

```
constexpr bool intersect(std::span<const T> lhs, std::span<const T> rhs) noexcept {
    const auto * a = lhs.data();
    const auto * b = lhs.data() + lhs.size();
    const auto * x = rhs.data();
    const auto * y = rhs.data() + rhs.size();
    return std::is_pointer_within(x, a, b)
        || std::is_pointer_within(y, a, b)
        || std::is_pointer_within(a, x, y)
        || std::is_pointer_within(b, x, y);
}

void transfer(std::span<const T> source, std::span<T> target)
    pre(not intersect(source, target) && source.size() == target.size())
{
    std::copy(source.begin(), source.end(), target.begin());
}
```

## defining total ordering is not feasible

---

Currently result of a comparison of two unrelated pointers is not specified in the language (according [\[expr.rel\]](#), but you can use `std::less<T *>` in the library [\[comparisons\]](#) for an implementation specified ordering). This proposal gives you ability to express a different thing than ordering: *is something related AND belongs inside?* That's in my experience more useful than global total ordering.

But what if you really want the total ordering for pointers? After all most of architectures are like that. For example [CHERI is different](#).

## can we make `std::less<T *>` work in constant evaluator

---

(Some) constant evaluators doesn't use real pointers for interpreted pointers, in order to be able to detect crimes against abstract machine and UB, (at least in clang) they consider of identifier of top-level object and provenance path. Unrelated objects there are just that ... unrelated symbols.

Hence `std::less<T *>` can't give us answer as there is no answer about relationship. But for proposed functionality the answer exists.

## static analysis and contracts

---

With this proposal we can specify relationship *belongs inside* to be usable for contracts and static analysis tools. For example we will be able to specify post contract "returned reference belongs to span you provided" without UB or implementation specific behavior.

## constexpr std::hive

---

For another example, `std::hive`'s member function `get_iterator(T *) -> iterator` checks all allocated blocks if provided pointer is in range, and then uses the right block's first member to calculate object's index to construct hive's iterator.

To make this constexpr we must have a way how to check without possibility of a failure to constant evaluate.

## Proposed function

---

```
constexpr bool is_pointer_within(const void * first, const void * p, const void * last);
```

If all pointers points to a same outermost enclosing object and `first <= p` and `p < last` result is true otherwise it's false.

## Why not just provide check for related pointers?

---

This is easy to implemented for constant evaluation, but not for runtime code. I think providing such functionality would provide false sense of security and lead to misuse, therefore I'm not proposing it.

## Why not span?

---

We don't have `std::span<const void>` and we `p` pointer can be pointing (or not pointing) to a subject of different type than the range (in case it would be typed) and this would lead to unnecessary template instantiation.

## Why it's [first, last) and not inclusive range?

---

It would be impossible to check for intersection of two ranges sitting next to each other. If there is demand for such functionality there is a possibility for `bool is_related_and_equal(const void * a, const void * b);`

Currently there is no way how to safely compare pointers for equality in constant evaluation if one of them points past the end.

## This signature seems error prone

---

I know, I was thinking if it should `first, last, pointer`, but without `span<const void>` or providing something like `struct memory_range { const void * start; const void * end; };` we will end up with three pointers anyway.

For me the most obvious order is to put them in expected order which will be the one which will return true, anything else would be confusing and users will be wondering if it was `first, last, pointer` or `pointer, first, last`, and will end up googling it, trying to open first link to [CPlusPlus.com](http://Cplusplus.com), noticing they are opening it, and quickly go back to open [C++ Reference](#) instead and still ends up waiting for fonts to load.

## Symmetry with `clamp`

---

Originally I had it `first, ptr, last`, but was told we have `std::clamp(ptr, first, last)`, so I'm mirroring existing signature. I don't want users to frantically google how it is supposed to be.

## `std::less<T *>`

---

Library specified implementation specific strict total ordering for `std::less<T *>`, but this is not a case for constant evaluated code.

## Weird or segmented architectures

---

On some platforms it's possible that the comparison return false positive, if somehow pointers points at the same memory but in different segments. This is already happening when someone tries to implement this with `std::less<T *>` and it's not a problem in practice as such architectures are rare ([16-bit x86 mode](#), [ESA/390](#), [z/Architecture](#)).

## Implementation

---

A prototype in Clang's [ExprConstant.cpp](#) and the new [bytecode interpreter](#), basically it needs a magic (consteval) compiler builtin to check if objects are related and use it to avoid failure which can't be recovered:

```
constexpr bool is_pointer_within(const void * ptr, const void * first, const void *
last) {
    if constexpr {
        // assuming existence of such builtin constant evaluation pointers
        // has all metadata needed to make this check easy
        if (!__builtin_pointer_related(first, ptr) || !__builtin_pointer_related(ptr, last)) {
            return false;
        }
    }
    return first <= ptr && ptr < last;
}
```

## Alternative way how to get same result

---

In cases where we have an array (or a set) of items, and wants to check if the item is inside array, I can just check against every single one for equality which is properly defined (unless one of the pointers is past-the-end pointer).

```
template <typename T> constexpr bool is_pointer_within(std::span<const T> array, const T * p) {
    // doesn't check past-end pointer
    return std::ranges::any_of(array, [=](const T & obj) { return std::addressof(obj) == p; });
}
```

Is this okay? It's not, it's O(n). Any such operation shouldn't be this slow. Also it doesn't allow us to check if subject is in the range as a subject, so the functionality is actually different.

Additional difference there is not defined comparison for checking unrelated pointer against past the end pointer.

## Wording

---

Add following to [\[memory.syn\]](#). Note wording is using what [\[expr.const\]](#) is already doing with comparison, and because objective is to avoid such failure I used same wording construct.

```
// [ptr.within], check if a pointer belongs to a specific memory range
constexpr bool is_pointer_within(const void * ptr, const void * first, const void * last); // freestanding
```

Add this somewhere in [\[memory\]](#) preferably after [\[ptr.align\]](#).

### 20.2.5 Pointer within pointers [ptr.within]

```
constexpr bool is_pointer_within(const void * ptr, const void * first, const void * last);
```

*Preconditions:* result of comparison `first <= last` is specified and is true ([expr.rel]) and `ptr` is not a past-the-end pointer.

*Returns:* true if comparisons `first <= ptr` and `ptr < last` are both specified and both are evaluated as true; otherwise false.

## Feature test macro

---

### 17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_is_pointer_within 2026??L // freestanding, also in <memory>
```